

Deep Dive — Judge Craft, Statistical Rigor, and Eval-Driven Development

Part 15 —  AI Evaluation · 15-ai-evals/notes/02-judge-craft-and-eval-driven-development.md

Note 01 gave you the vocabulary and the pyramid. This note is the craft: how to actually build a rubric that means something, how to know whether your judge is measuring your rubric or measuring noise, how to not fool yourself with a p-value, and how to wire all of it into a workflow that makes your system better over time instead of just producing dashboards nobody trusts.

1. LLM-as-judge, in depth

A judge is a **model plus a rubric**, and the rubric is the product. Everything about judge quality traces back to rubric design.

1.1 Rubric construction

```
BAD "Rate the helpfulness of this answer from 1 to 10."

GOOD "Score each criterion independently. Output JSON.
      grounded: every factual claim appears in CONTEXT (0 or 1)
      complete: addresses every part of the question (0 or 1)
      concise: no preamble, no restating the question (0 or 1)
      safe: no promises beyond the published policy (0 or 1)
      Then: reasoning (<=2 sentences), then the JSON object."
```

Four design rules, in order of how much damage skipping them does:

1. **Binary or small-ordinal criteria, not a wide scale.** Neither models nor human annotators can reliably distinguish a 6 from a 7 on a 1-10 scale — you end up averaging noise and calling it signal. A criterion that's satisfied or not is a question both graders can actually answer the same way twice. `code/judge_bias_and_calibration.py` demonstrates this quantitatively: on identical simulated data with identical judge biases, a binary rubric reaches **moderate** agreement with human labels (kappa ~ 0.5) while a holistic 1-10 score reaches only **slight** agreement (kappa ~ 0.1) on the decision-relevant, exact-match reading — the number that matters if you're gating CI on it.
2. **Reasoning before the score, always.** Asking for a one-line justification before the number measurably improves judgments (the model has to commit to *why* before committing to *what*), and it gives you something to read when you disagree with a score. A bare `0.4` with no reasoning is not an eval result, it's a rumor.
3. **Name what "satisfied" means, concretely, in the criterion itself.** "Complete: addresses every part of the question" is checkable. "Good: is a good answer" is not a rubric, it's a restatement of the question you're trying to answer.

4. **One rubric change at a time, versioned.** A rubric is a prompt, and prompts are code. Treat rubric edits like you'd treat a change to a scoring function in a Module 01 pipeline — diffed, reviewed, and re-run against your calibration set before it ships (§2 below).

1.2 The bias catalog

Bias	What it looks like	Mechanism	Mitigation
Position bias	In pairwise comparison, the judge prefers whichever answer it saw first — on identical content	Attention/recency and training-data order effects	Randomize order; score both orders and average
Verbosity bias	Longer answers score higher regardless of correctness	Length correlates with "thoroughness" in training data the judge itself was tuned on	An explicit concise/no-preamble criterion, isolated from other criteria — see the decomposition in <code>code/judge_bias_and_calibration.py</code> §2
Leniency drift	Judge scores cluster at the top of any scale (most scores land at 0.8+), destroying resolution	Judges are tuned to be helpful and non-confrontational; "satisfied" is the path of least resistance	Track the rate of "satisfied" on criteria you KNOW should fail on a held-out negative set; recalibrate the rubric prompt when that rate creeps up
Self-preference	A model rates outputs from its own family higher	Shared stylistic priors from similar training data/RLHF	Judge with a model from a different family than the one under test
Format bias	Bullet points and headers score above equally-good prose	Structure reads as "effort"	Explicit content-only criteria; strip formatting before judging if format isn't part of the spec

None of these are exotic edge cases — they show up in the first calibration run of almost every judge anyone builds. Budget time for this, don't discover it in production three months later when someone asks why the eval says 92% and users are furious.

1.3 Calibrating a judge against humans (the step everyone skips)

1. Take 50-100 examples, ideally from your real failure taxonomy (note 01 §4), not hand-picked easy ones.
2. Have a human score them with **your exact rubric** — not a vibe check, the same JSON schema the judge will produce.
3. Run the judge on the same examples, same rubric, temperature 0 (you want the judge's *systematic* behavior, not its sampling variance, for this step).
4. Compute Cohen's kappa (§1.1 above explains why unweighted, for a binary/small-ordinal rubric, is the number to gate on — see `code/judge_bias_and_calibration.py` for the formula, self-checked against two hand-solved reference values).
5. **Below ~0.6-0.7 kappa, the judge is not measuring your rubric — it's measuring something correlated with it, loosely.** Fix the *rubric prompt* first: add a missing criterion, add a worked example of a borderline case, remove language that invites leniency ("try to be generous"). Only reach for a different judge model after the prompt has had a real iteration or two.

6. **Re-check quarterly and after every judge-model upgrade.** A judge is a model; models drift, and a provider's silent model-version bump can silently shift your judge's leniency without anyone touching your rubric.

Write the kappa number down somewhere permanent — a model card, a runbook, a comment in the eval config. "We calibrated the judge once, eighteen months ago" is not calibration, it's an artifact.

1.4 Pairwise vs. absolute scoring

"Is this a 7 or an 8" is hard for humans and models alike. "Is A better than B" is comparatively easy for both — which is the entire case for pairwise comparison.

	Absolute (rubric score)	Pairwise (A vs. B)
Best for	Hard constraints, regression gates, anything with a fixed bar	Prompt/model A-B tests, style and tone, "does this feel better"
Weakness	Scale compression, leniency drift	Gives you a <i>relative</i> answer only — both candidates can be terrible
Position bias exposure	N/A	Real, and must be mitigated by randomizing/averaging order

Do not use pairwise for hard constraints. "Did it leak a credit card number" is pass/fail, not a preference. Averaging a constraint into a pairwise preference score is how constraints get shipped broken — keep at least one absolute floor metric alongside any pairwise comparison.

2. Statistical rigor, in depth

An eval score is a sample statistic estimating an unknown true quantity, exactly like a poll. Treat it as a fact and you will ship phantom regressions, chase phantom wins, and eventually stop trusting your own eval suite — rightly, because you earned that distrust by not doing this section.

`code/statistical_rigor_for_evals.py` builds and formula-checks all four pieces below; read it side by side with this section.

Confidence intervals on a proportion. An eval accuracy IS a proportion (k correct out of n). The naive interval ($\hat{p} \pm z \cdot SE$, the "Wald" interval) collapses to zero width at $\hat{p} = 0$ or 1 and has poor coverage at small n — exactly the regime most eval slices live in. The **Wilson score interval** fixes both problems and should be your default; it's a few more lines of arithmetic and there's no reason not to use it.

McNemar's test for paired comparisons. Two prompts scored on the *same* eval set are correlated observations, not independent samples — both get the easy examples right and stumble on the same hard ones. An unpaired test throws that correlation away and is systematically underpowered. McNemar's test looks only at the **discordant pairs** (where the two prompts disagree) and asks whether the split between "A right, B wrong" and "A wrong, B right" looks like a coin flip. Below ~ 25 discordant pairs, use the exact binomial form; above it, the continuity-corrected chi-square approximation is standard.

Paired bootstrap for continuous scores. McNemar needs binary correct/incorrect. Rubric sums, RAGAS-style [0,1] metrics, and judge scores are continuous — resample **example indices** (not the two score arrays independently) so the pairing survives into every bootstrap resample, then read the percentile interval on the mean paired difference.

Sample size is not a detail. The same 72% vs. 74% accuracy gap is statistically invisible at $n=50$ (confidence intervals swallow each other, McNemar can't reject "these are the same prompt") and unmistakable at $n=2000$, for the *exact same underlying two accuracies*. There is no universal "n is enough" number — compute the power for the delta size you actually care about, using the sizing table in note 01 §4 as a starting point, not a substitute.

p-hacking your own eval set. If you re-run a fixed, small eval set against many prompt variants and keep whichever looks best, some fraction of "wins" are pure sampling noise even when every variant is identical in true quality — the script demonstrates this directly by generating variants with zero real difference and counting how many look "significant" anyway. This is the same overfitting pathology as tuning hyperparameters against a small validation set (Module 01), except the knob is prompt wording. Fix: hold out a locked test slice you touch rarely, widen the eval set, and correct for multiple comparisons — or at minimum, distrust any p-value produced by a search over many candidates.

The rule that saves you the most future embarrassment: never report a bare accuracy number in a place someone will make a ship/no-ship decision from it. Report the interval, and if it's a paired comparison, report the paired test's p-value next to it.

3. Evaluating specific system types (pointers, not repeats)

Each system type has its own metric vocabulary, and repeating it here would duplicate modules that already own it properly. Use this as a map, not a summary:

System type	What's different about evaluating it	Where it's covered
RAG	Faithfulness/groundedness, context precision & recall, answer relevance — metrics that separate "the retriever found the right thing" from "the generator used it correctly"	<code>../08-rag/</code> (RAGAS and friends)
Agents	Trajectory correctness (did it take the right steps, not just reach the right answer), tool-selection accuracy, compounding per-step error	<code>../06-ai-agents/notes/02-planning-failure-and-evals.md</code>
Safety / security	Adversarial and red-team suites, jailbreak and injection resistance, refusal-rate tuning (both directions)	<code>../13-ai-security/</code>
Multi-agent / agentic systems	Emergent failure modes across hand-offs, on top of everything single-agent evaluation already needs	<code>../07-agentic-ai/</code>

The underlying statistical discipline in §2 above — intervals, paired tests, honest sample sizing — applies identically regardless of which domain-specific metric you're computing. Don't let a fancy metric name excuse a bare number with no interval.

4. Online evaluation: the production half

Offline evaluation needs reference outputs and a fixed dataset. Production has neither. **Online evaluation** runs reference-free evaluators (usually LLM judges, sometimes simple heuristics) over live traffic: groundedness, policy compliance, refusal appropriateness, "did it even answer the question."

Three knobs make or break an online evaluation setup:

- **A filter** — which traffic gets judged. Everything, negative-feedback traffic, a random sample, a specific customer tier. Judge what you actually need signal on, not everything by default.
- **A sampling rate** — judging 100% of production doubles your inference cost on every request. 5-10% of healthy traffic plus 100% of anything with negative feedback is a reasonable starting shape.
- **A spend cap**. An online judge is an unbounded cost loop attached to your live traffic. Set an actual weekly limit; traffic spikes are exactly when you'd otherwise get an unpleasant invoice.

Guardrail metrics vs. quality metrics. A guardrail metric (PII leakage rate, policy-violation rate, refusal-on-benign-request rate) is a floor you alert on immediately, the same way you'd alert on an error rate. A quality metric (helpfulness, tone) is a trend you watch, not a pager-worthy event on a single bad sample. Conflating the two means either your pager fires on noise or your real incidents get lost in a quality dashboard nobody has a pager for.

A/B testing an LLM change is still A/B testing — randomize assignment, pre-register the metric and the minimum detectable effect, and apply the exact same paired-comparison discipline from §2 to the online results. The fact that the system involves a prompt instead of a button color changes nothing about the statistics.

5. Human evaluation, done well

Human eval is the most expensive tier and the one everything else calibrates against — spend it deliberately, not as an afterthought.

- **Structured beats freeform.** A rubric with named, checkable criteria (the same rubric your LLM judge uses) produces comparable, aggregable results. "Any thoughts?" produces a paragraph of vibes per reviewer that doesn't aggregate into anything.
- **Measure inter-annotator agreement**, not just the average score. Two humans disagreeing with each other as much as your judge disagrees with either of them means your "human ground truth" isn't ground truth — it's a third noisy opinion, and your rubric probably has an ambiguous criterion.

- **Multiple reviewers per item**, at least on your calibration set, specifically so you *can* measure agreement. A single-reviewer pipeline has no way to notice its own rubric is broken.
- **Route the highest-value human attention to disagreement and uncertainty**, not to re-reviewing everything the judge is already confident and correct about. Human time is the scarcest resource in this whole pyramid — spend it where the judge is least reliable.
- **A named human reads the queue on a schedule**. An annotation queue nobody reads is a to-do list, not an evaluation process.

6. Eval-driven development: the workflow, as a capstone

Everything above is a set of parts. This is how they assemble into something that makes a system better over time instead of producing dashboards nobody trusts.

```
production traffic
|
v
negative feedback / errors / spot-checks  <-- note 01 SS4: harvest real failures
|
v
add to golden dataset, tag with the failure taxonomy category
|
v
fix the system (prompt, retrieval, rubric)
|
v
re-run the FULL suite: deterministic -> judge (calibrated, SS1) -> human spot-check
|
v
paired significance test against the PINNED baseline (SS2) -- not a bare delta
|
v
CI gate: absolute floors AND regression deltas, both must pass
|
v
ship -- and the dataset that caught the last regression is now permanent
|
+-----> back to production traffic
```

The properties that make this a *ratchet* instead of a one-time report:

- **The dataset only grows, and is versioned**. Every regression you catch becomes a permanent test case. A dataset that doesn't grow isn't being fed by real failures — check whether step 1 is actually wired up.
- **CI gates on both an absolute floor and a regression delta**. A floor alone lets quality erode slowly as long as it stays above the line; a regression delta alone lets you ship something bad as long as it's not worse than yesterday's bad. You need both.
- **The baseline is pinned, not "whatever's currently deployed"**. Compare against a fixed, named version/commit so a delta means something specific, not "different from whenever someone last looked."
- **Evals are a living asset with an owner**, not a one-time report generated for a launch review. The rubric gets revised, the dataset gets pruned of examples that turned out to be mislabeled, the judge gets

recalibrated on schedule (§1.3). A suite nobody maintains decays exactly like a benchmark decays (note 01 §1) — just faster, because you're the only one who was supposed to be watching it.

`../16-ai-tech-stack/tracks/langsmith/notes/02-evaluation-and-operations.md` implements this exact loop end to end with a real tool (datasets, `evaluate()`, CI gates, online evaluators, annotation queues) — read it once you can name every step in the diagram above without looking at it.

7. Where this returns

Idea here	Where it returns
Binary rubric > Likert judge, with kappa	<code>code/judge_bias_and_calibration.py</code> — full simulation and numbers
Wilson CI, McNemar, paired bootstrap, p-hacking	<code>code/statistical_rigor_for_evals.py</code> — formulas, self-checks, demos
Judge calibration workflow	<code>../16-ai-tech-stack/tracks/langsmith/notes/02-evaluation-and-operations.md</code> SS3
RAG-specific metrics	<code>../08-rag/</code>
Agent trajectory evaluation	<code>../06-ai-agents/notes/02-planning-failure-and-evals.md</code>
Red-team / adversarial suites	<code>../13-ai-security/</code>
Audit trails for eval results and sign-off	<code>../12-ai-governance/</code>
CI gates, dataset versioning, online evaluators, annotation queues	<code>../16-ai-tech-stack/tracks/langsmith/</code>